# NyayaSetu AI

Autonomous Legal Advisory & Statutory Complaint Drafting System

## PART 1: COMPREHENSIVE PROJECT DESCRIPTION

**Executive Summary:** NyayaSetu AI is an end-to-end legal intelligence platform built to assist victims, compliance officers, and legal counselors in navigating sensitive statutory frameworks in India—specifically the **PoSH Act, 2013** (Prevention of Sexual Harassment at Workplace) and the **POCSO Act, 2012** (Protection of Children from Sexual Offences). The system transforms raw, distressed, colloquial narratives into court-admissible, strictly formatted legal complaint documents ready for Internal Committees (IC) or law enforcement.

### System Architecture & Core Pipeline

**1. Frontend Layer:** Responsive Single Page Application (SPA) built with Vanilla JavaScript (ES6+) and modern custom CSS design tokens, providing progressive analysis and live complaint editing.
**2. PII Sanitization Layer:** Bidirectional pseudonymizer that scrubs personal names, phone numbers, and addresses before cloud model dispatch, restoring verified identities locally.
**3. Statutory Classification:** High-confidence evaluator categorizing claims into PoSH, POCSO, or Out-of-Scope with statutory rationale and mandatory Section 19 alerts for minors.
**4. Structured Fact Extraction:** Hybrid LLM JSON extraction (Groq Qwen 2.5) paired with deterministic regex fallbacks for dates, locations, designations, and witness identification.
**5. Statutory RAG Engine:** ChromaDB vector database backed by *all-MiniLM-L6-v2* embeddings indexing legal corpora to retrieve authoritative statutory citations.
**6. Document Generation:** High-fidelity Microsoft Word (.docx) generation with strict legal styles, 3-column witness matrices, and solemn statutory declarations.

| Domain Layer | Technologies & Libraries |
|---|---|
| Backend Framework | Python 3.11+, FastAPI, Uvicorn, Pydantic v2 |
| AI / LLM Engine | Groq Cloud API (Qwen 2.5 / Llama 3.3 in JSON Mode), Ollama Fallback |
| Vector DB & RAG | ChromaDB, sentence-transformers (all-MiniLM-L6-v2), LangChain |
| Document Pipeline | python-docx with XML custom cell borders & typography control |
| User Interface | Vanilla HTML5, Modern CSS Design System, Modular JavaScript ES6+ |

## PART 2: HIGH-IMPACT RESUME BULLET POINTS

Copy and paste these directly into your resume's Experience or Projects section:

- **Architected NyayaSetu AI**, an end-to-end legal advisory platform converting unstructured incident narratives into statutory-compliant complaint drafts under Indian **PoSH (2013)** and **POCSO (2012)** Acts.
- **Built a High-Precision Fact Extraction Pipeline** combining **LLM JSON Mode** (Groq / Qwen 2.5) with regex heuristic engines, achieving 100% data extraction for parties, dates, locations, and witnesses even from unpunctuated colloquial text.
- **Integrated a Retrieval-Augmented Generation (RAG) System** utilizing **ChromaDB** and *all-MiniLM-L6-v2* embeddings to ground draft complaints with exact statutory citations, legal sections, and automated mandatory reporting alerts.
- **Engineered Automated Word (.docx) Legal Exporter & PII Anonymizer**, sanitizing sensitive personal data before model inference and generating structured, court-admissible complaint documents with dedicated witness and

declaration tables.

# PART 3: INTERVIEW PREPARATION GUIDE

## 1. The 60-Second Elevator Pitch

*"I developed **NyayaSetu AI**, an AI-powered legal advisory and automated complaint drafting agent. Filing legal complaints under complex Indian frameworks like the PoSH Act (workplace harassment) and POCSO Act (child protection) is intimidating, and victims often provide fragmented or distress-filled narratives. My system takes this raw text, classifies the legal scope, extracts critical facts (parties, dates, timeline, location, witnesses) using a hybrid LLM JSON pipeline with deterministic fallbacks, retrieves relevant legal sections using ChromaDB RAG, and produces a court-ready, formatted Word document. I also built a PII-masking layer to preserve complainant confidentiality."*

### Q1: Why did you use a hybrid LLM + Regex approach instead of just prompting an LLM?

In legal domain applications, deterministic accuracy and zero hallucination are essential. When users input short, panicked, or poorly formatted text (e.g. 'my name ravneet kaur my manager at hanuai sexual harass me on 5 oct 2026 witness is harish'), smaller or quantized LLMs can misattribute roles or miss entities due to lack of punctuation. By implementing a hybrid approach—prompting the LLM with strict JSON schemas first, and passing the output through regex fallback normalization—we guarantee accurate data extraction for names, designations, dates, and witnesses even if the LLM connection fails or returns partial JSON.

### Q2: How does your RAG pipeline work, and what problem does it solve?

The RAG pipeline solves legal hallucination. LLMs often invent legal sub-clauses or cross-contaminate PoSH and POCSO sections. I chunked the official PoSH Act 2013 and POCSO Act 2012 statutes, embedded them using sentence-transformers/all-MiniLM-L6-v2, and indexed them in ChromaDB. When an incident is analyzed, the system queries ChromaDB using the incident summary and category, retrieves the top-k statutory sections, and passes them as grounding context into the draft generator.

### Q3: How do you handle user privacy and PII?

Privacy is paramount, especially under Section 16 of the PoSH Act and Section 33(7) of POCSO, which strictly prohibit disclosing identities. Before sending narrative text to the cloud LLM, our regex-based PII anonymizer detects names, phone numbers, and emails and maps them to placeholders like [Complainant_1]. The inference runs on the masked text, and once the draft is received, the de-anonymizer restores the original entities locally before rendering or generating the DOCX.

### Q4: What was the most challenging technical edge case you solved?

A major challenge was entity bleed-over in unstructured input. For example, when a user wrote 'my manager at hanuai', the model or regex initially captured 'At Hanuai' as the manager's name rather than recognizing 'manager' as a designation and 'Hanuai' as the location/company. I resolved this by refining the regex boundary rules, enforcing stopword filters for prepositions (at, in, on), standardizing missing respondent names to '[Name Withheld / Unknown - ]', and synchronizing the frontend payload directly with the backend DOCX table generator.